cuDESeq2: GPU acceleration of the DESeq2 differential-expression pipeline

Max Highsmith [1,*], Greg Koytiger [1,*]

[1]Synthesize Bio

[*]To whom correspondence should be addressed: maxrhighsmith@gmail.com; greg@synthesize.bio

Intended article type: *Original Paper*. Subject: Gene expression.

Abstract

Motivation: DESeq2 is widely used for bulk RNA-seq differential-expression analysis, but its CPU-bound per-gene dispersion and generalized linear model fits make repeated analyses costly.

Results: We present cuDESeq2, a PyTorch reimplementation that batches gene-wise computation on the GPU while following the reference workflow. Against R DESeq2 1.52.0 across six real designs (7–300 samples, $P = 2$–5), final dispersion has a median p95 relative error of 8.5e-4, significant-gene sets have Jaccard similarity of at least 0.99963, and shrunken LFCs have Pearson correlation of at least 0.99722. The standard Wald path includes count-outlier replacement and refitting. On one A100, cuDESeq2 is 2.9–13.4× faster end-to-end than the faster of one-worker and 12-worker R.

Availability and implementation: Open source under the MIT license at `https://github.com/synthesizebio/gpu_deseq`; PyTorch and a CUDA GPU are required for the reported acceleration.

Contact: maxrhighsmith@gmail.com; greg@synthesize.bio

Supplementary information: Appended to this manuscript.

1 Introduction

DESeq2 (Love et al., 2014) is a standard method for bulk RNA-seq differential-expression analysis. A typical analyis inlcudes 5 steps: normalization, dispersion estimation, fitting a negative-binomial GLM, significance testing, and Log fold change shrinkage estimation (Zhu et al., 2019). Many components of the dispersion, GLM and shrinkage calculations are iterative and independent by gene. In its current implementation, DESeq2 performs these calculations on the CPU with its parallel mode distributing blocks of genes among workers (Fig. 1).

Historically, wall time is a secondary concern for a single DE analysis because the pipeline is only performed a handful of times per project. However, wall time becomes more important when DE must be repeated 1000s ot times as part of an evaluation loop or comparison of model performance. The bioinformatics community has recently seen growing interest in virtual-cell systems and foundation models for gene-expression data (Theodoris et al., 2023; Cui et al., 2024; Bunne et al., 2024; Adduri et al., 2025; Koytiger et al., 2025). DE is a natural tool for evaluation of such models, and the need to repeat DE analyses in a loop has become more common. Some recent benchmarks have limited wall time by utilizing rank-based tests on normalized expression as less expensive surrogates (Roohani et al., 2025; Wei et al., 2026). A faster implementation of the DESeq2 model would permit rigoruous and comprehensive reference analysis to remain in that an evaluation loop.

The established route to a faster DESeq2 thus far has been algorithmic and CPU-bound. glmGamPoi (Ahlmann-Eltze and Huber, 2020) accelerates Gamma–Poisson GLM fitting on large, sparse count matrices. However, replacing DESeq2 with glmGamPoi changes the inferential workflow and forgoes DESeq2's automatic outlier replacement, limiting comparability with established DESeq2 results and removing a safeguard against influential counts. PyDESeq2 (Muzellec et al., 2023), a Python reimplementation, improves interoperability and speed on large datasets, though by its authors' own account it yields results "similar, but not identical" to DESeq2; GPU acceleration has meanwhile transformed adjacent single-

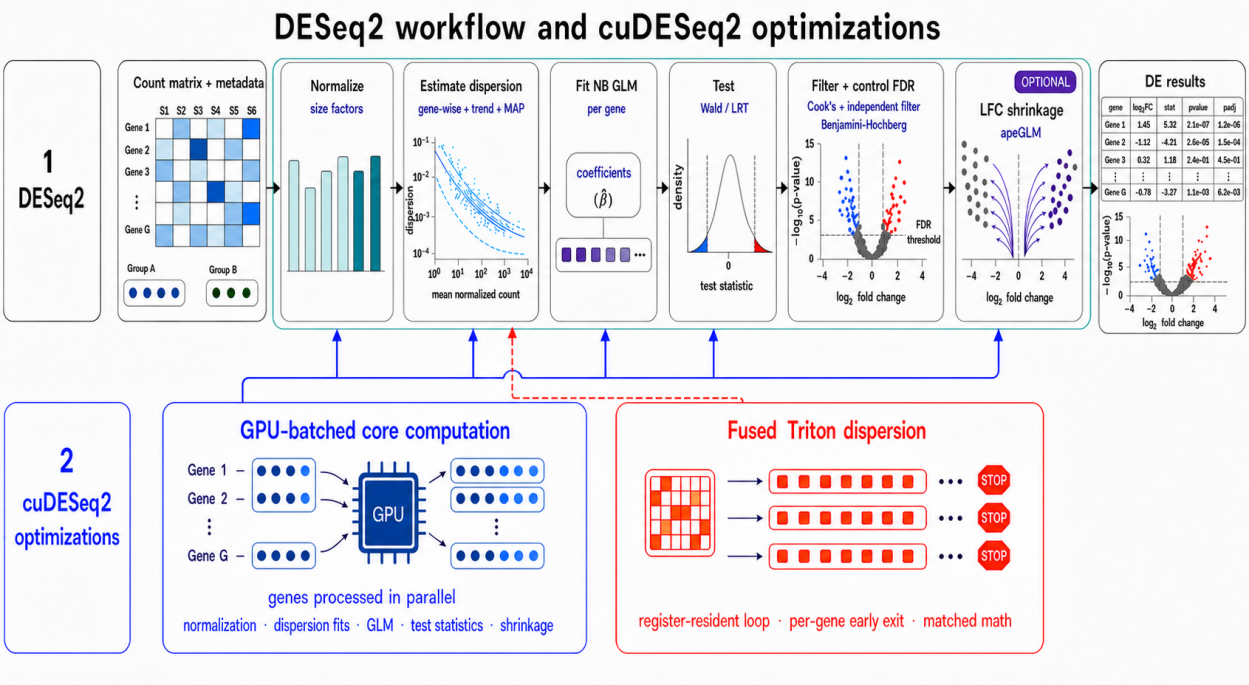

Figure 1: Overview of cuDESeq2. *Top*, the five stages of a DESeq2 run. *Bottom*, where the GPU work goes: DESeq2's per-gene, sequential CPU work (dispersion estimation, the negative-binomial GLM, and apeGLM LFC shrinkage) is batched across all genes (*left*, solid arrows), and the dispersion fit, which is limited mainly by launch latency, has a fully fused Triton implementation (*right*, dashed arrow). A third mode uses CUDA-graph chunked replay; the three modes are compared in Table 1. Every stage is checked against R DESeq2 step by step (Sec. 3.2).

cell workflows, where rapids-singlecell and ScaleSC report 15–20× end-to-end speedups (Hu et al., 2025), but those gains come from the dense and sparse linear-algebra core: PCA, neighbor graphs, clustering.

In this paper we present the following contributions

(i) Batched GPU implementations of DESeq2's gene-wise dispersion, GLM, and apeGLM shrinkage calculations, including a reference-compatible L-BFGS optimizer that keeps LFC shrinkage on the GPU.

(ii) A CUDA-graph mode (chunked replay) that removes launch overhead from the dispersion loops, using a capture-safe no-pivot LU in place of cuSOLVER.

(iii) A fully fused Triton kernel: one program per gene runs the whole gradient-ascent loop

in registers with per-gene early exit.

(iv) End-to-end and per-step timing benchmarks against R across six real-data designs, together with sample-count scaling measurements for the three GPU execution modes.

2 Methods

2.1 Reference workflow and supported scope

DESeq2 (Love et al., 2014) models each gene independently with a negative-binomial GLM. For gene g with counts K_{gi} over samples $i = 1, \ldots, S$, design matrix $X \in \mathbb{R}^{S \times P}$, and size factors s_i, it assumes $K_{gi} \sim \mathrm{NB}(\mu_{gi}, \alpha_g)$ with $\mu_{gi} = s_i \exp(x_i^\top \beta_g)$ and variance $\mu_{gi} + \alpha_g \mu_{gi}^2$. The pipeline is composed of five stages: Normalization, Dispersion Estimation, GLM Fit, Significance Testing, and LFC Shrinkage.

2.1.1 Normalization

Median-of-ratios normalization is implemented with GPU tensor reductions and quantiles, including an automatic fallback based on DESeq2's optional positive-counts estimator.

2.1.2 Dispersion estimation

Gene-wise dispersion fitting for both the MLE and MAP estimators is batched across genes. The optimization methods used for this stage are described in Sec. 2.2.

2.1.3 GLM fitting and hypothesis testing

Negative-binomial IRLS fits and Wald or likelihood-ratio tests are batched over genes; the implementation details are given in Sec. 2.3.

2.1.4 Significance filtering and result assembly

Cook's filtering, independent filtering, and multiple-testing adjustment are implemented as vectorized array operations. The standard wrapper also performs count replacement and refitting when DESeq2's eligibility conditions are met.

2.1.5 LFC shrinkage

The apeGLM optimizer is batched across genes on the GPU; Sec. 2.4 describes the implementation.

2.2 Dispersion acceleration

The dispersion stage is a batched port of DESeq2's `fitDisp` and the supporting `estimateDispersions` logic. Its gene-wise MLE maximizes the Cox–Reid adjusted profile log-likelihood

$$\ell_{\mathrm{CR}}(\alpha_g) = \sum_{i=1}^{S} \log \mathrm{NB}\big(K_{gi}; \mu_{gi}, \alpha_g\big) \; - \; \tfrac{1}{2} \log \det\big(X^\top W_g X\big), \tag{1}$$

where $W_g = \mathrm{diag}\big(\mu_{gi}/(1+\alpha_g\mu_{gi})\big)$. We retain DESeq2's initialization, line search, trend fit, MAP shrinkage, and outlier rule. Becuase dispersion is a signifcant source of the pipelines computation time we expand optimization beyond basic batching and provide three alternative implementations of the gradient-ascent loop: earger, CUDA-graph, and Triton. The three implementations use the same optimizer decisions and differ only in how those evaluations are scheduled.

Table 1: The three execution modes for the dispersion loop.

Mode	Dispatch	Fidelity	Designs
eager	one kernel per tensor op	reference	any P
graph	replay a captured CUDA graph	bit-identical to eager	any P
Triton	one fused kernel per gene	R parity, not bit-identical	$P \in \{2, \ldots, 6\}$

Both accelerated modes fall back to eager on CPU; Triton also falls back for $P = 1$ or $P > 6$. Triton's agreement with eager is quantified in Sec. 3.2.

CUDA-graph chunked replay. In eager mode each gradient-ascent iteration launches a handful of tiny kernels over 1–10 MB arrays, so the loop is dominated by launch latency. Capturing the loop as a CUDA graph amortizes that cost into a single replay, but two obstacles must be overcome.

The first is early exit. A fixed full-length capture would force every gene to the maximum iteration count, which wastes the typical 15–40-iteration case. To prevent this we capture a K=10-iteration graph that reads and writes a set of persistent state buffers in place, and replay it in a loop with a convergence check between chunks. A chunk always runs to its boundary, so a gene that converges partway through executes further iterations. We selected ten iterations per replay empirically to balance replay overhead against excess iterations after convergence.

The second is cuSOLVER. The Cox–Reid term needs $\log\det(b)$ and $\mathrm{tr}(b^{-1}\,\mathrm{d}b)$ with $b = X^\top W X$, but `slogdet`/`solve` host-synchronize and cannot be captured. Since b is symmetric positive-definite for full-rank X, we replace them with an unrolled no-pivot LU (`_logdet_and_tr_binv_db`) returning both quantities from elementwise and indexing operations alone, matching the `linalg` path to ∼1e-15 for $P = 2 \ldots 6$.

Before capture the chunk is run three times on a side stream, warming the caching allocator and any lazy kernel initialization. Captured graphs are cached on G, S, P, the chunk length K, whether a prior term is enabled, the dtype and the device. Only *whether* there is a prior is selected on the device, because the prior variance lives in the buffer set as a scalar tensor the replay reads, so one graph serves datasets whose prior variances differ. Because the graph uses the same routine, it retains bit-identical parity with the eager mode implementation.

Triton full-loop fusion. Each Triton dispersion fit uses one kernel launch over a grid with one program instance per gene (Fig. 2). Each instance loads its counts, μ, and the design columns once into registers, then runs the iterative gradient-ascent loop register-resident: the

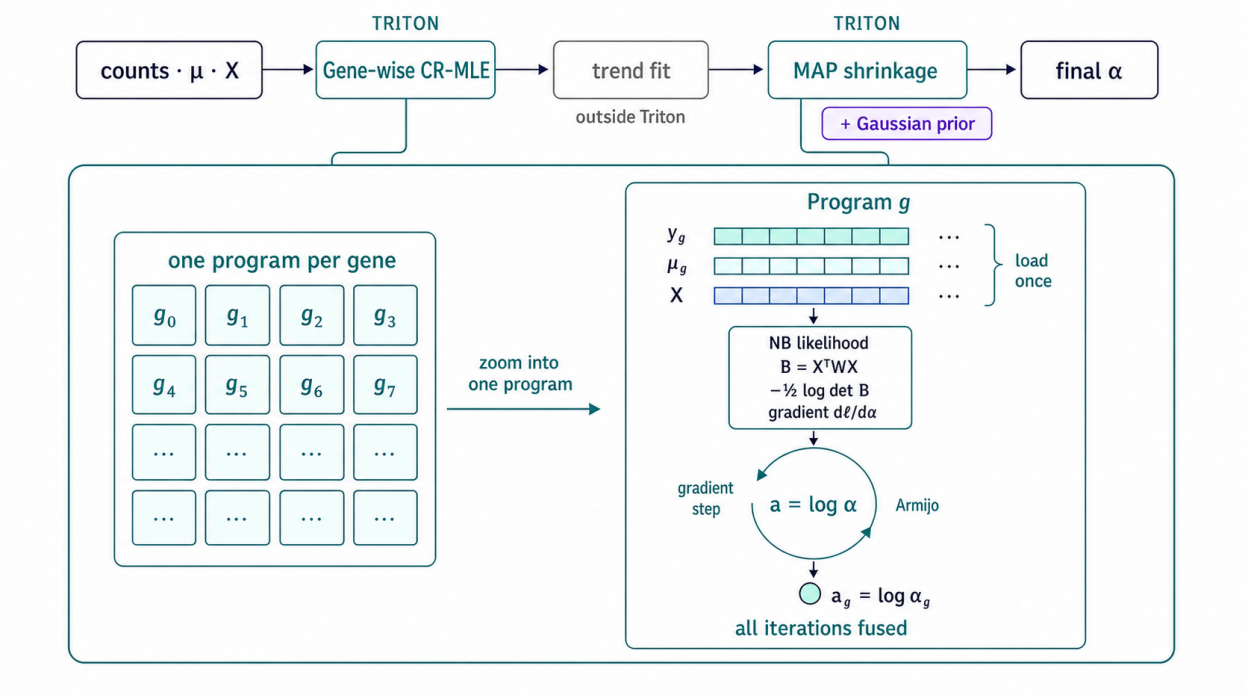

Figure 2: The fused Triton dispersion kernel. The same kernel implementation is invoked separately for the two gene-wise fits of the dispersion pipeline: the Cox–Reid MLE and the MAP shrinkage with a Gaussian prior that follows the trend fit (the trend fit itself runs outside Triton). Each invocation is one kernel launch over a grid with one Triton program instance per gene. Each instance loads $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and the design $\mathbf{X}$ once, then runs every iteration of the gradient ascent register-resident, with the NB log-likelihood, the Cox–Reid term $-\frac{1}{2}\log\det\mathbf{B}$ ($\mathbf{B} = \mathbf{X}^\top\mathbf{W}\mathbf{X}$), the $\log\alpha$-gradient and the Armijo line search all fused into the loop body. Only the converged $a_g = \log\alpha_g$ is written back to global memory. Supplementary Algorithms S1–S2 give the pseudocode.

Cox–Reid $P \times P$ solve (closed-form 2×2 for P=2, an unrolled no-pivot LU above that), the NB log-likelihood and $\log \alpha$-gradient (`lgamma` and `log1p` from `libdevice`, `digamma` hand-implemented to match ATen's `calc_digamma` through a recurrence to $x \geq 10$ followed by the Bernoulli asymptotic series), and the Armijo line search. A per-gene `while` loop lets each gene exit as soon as it converges, so the kernel does not run every gene to the batch maximum. The kernel is fp64 and covers $P \in \{2, \ldots, 6\}$; `fit_dispersions` gates on the covered range and routes wider designs to the eager path, while the kernel's own launcher raises instead of fitting a truncated model, since it reads a fixed number of design columns. At the gene-wise step, non-converged and high-dispersion genes get the same noIncrease revert and grid fallback as the eager path, computed eagerly after the kernel returns; at the MAP step neither applies, in either mode, because R applies neither. The fused reduction order and the hand-written `digamma` move the last bits, so this mode is not bit-identical to eager, and Sec. 3.2 reports how far the two diverge on real data.

2.3 GLM acceleration

The GLM is implemented as a batched negative-binomial IRLS solve on the GPU, using one batched factorization across genes at each iteration. Genes that diverge or reach the DESeq2 iteration cap use the same host-side fallback as the reference. Wald standard errors retain the $\mu \geq 0.5$ floor in `fitBeta.cpp`; LRTs use the full–reduced deviance difference.

2.4 LFC shrinkage acceleration

apeGLM (Zhu et al., 2019) is a per-gene optimization, so its L-BFGS state, line search, and convergence flags are batched on the GPU. We match the reference optimizer's initialization and stopping rules because extreme, low-count genes can have competing posterior modes. As in `lfcShrink`, only the shrunken coefficient and its standard error are replaced; p-values remain those from the Wald fit.

2.5 Validation methodology

Reference fixtures are generated by `scripts/generate_r_fixtures.R` against R DESeq2 1.52.0: three single-factor designs at 12×200, 30×500 and 60×2000 (samples $\times$ genes, $P = 2$), a 60×1500 two-factor design $\sim$ batch + condition with a three-level batch ($P = 4$), and a 60×1000 continuous-covariate design ($P = 2$). Every intermediate is checked against R: size factors, `dispGeneEst`, `dispFit` (the parametric trend on every fixture, plus the `"local"` and `"mean"` trends on the single-factor ones), `dispMAP`, the final dispersion and its outlier set, β, μ, the hat-matrix diagonals, Cook's distances, Wald SE, the LRT statistic on all five designs, the independent-filtering `padj`, and the apeGLM prior scale and shrunk LFC/SE. Errors are reported as p95 relative values, or absolute values where a quantity crosses zero. Sec. 3.2 reports the measured agreement for the principal quantities and tabulates the five pipeline stages on the real datasets (Table 2).

3 Results

3.1 Experimental setup

All GPU measurements are taken on one NVIDIA A100-SXM4-40GB under PyTorch 2.6 with CUDA 12.4 and NVIDIA driver 550.90.07. The reference CPU baseline (host A) is R DESeq2 1.52.0 on a 12-vCPU virtual machine, with R 4.6.0 and apeglm 1.34.0. Every timing is a warm median of five repetitions with `torch.cuda.synchronize()` on both sides of the timed region, so no GPU work is left in flight when the clock stops. End-to-end R timings use both one worker and 12 BiocParallel workers; Table 3 uses the faster configuration for each design. Table 3 times the ordinary `DESeq()` call path, whereas Table 4 times an output-equivalent staged call path in a separate session so that the five stages can be measured individually.

We validate on three real, published RNA-seq datasets sliced into six designs spanning

$P = 2$–5 model terms: pasilla (Brooks et al., 2011) (*Drosophila*, n=7, one- and two-factor), airway (Himes et al., 2014) (human, n=8, three design slices), and a 300-sample GTEx (GTEx Consortium, 2020) whole-blood vs. skeletal-muscle contrast (54,922 genes). The first two are distributed as Bioconductor experiment packages; the GTEx counts are recount2's uniform reprocessing of SRA study SRP012682 (Collado-Torres et al., 2017), from which we draw 150 samples per tissue at a fixed seed and convert base coverage to read counts by dividing by the average read length. Scaling studies use fixed-seed negative-binomial simulations where the gene and sample axes can be swept independently.

3.2 Numerical parity with R

We compared each stage of the pipeline with R DESeq2 1.52.0 across six real-data designs derived from three datasets (Table 2). Agreement was evaluated separately for normalization, dispersion estimation, GLM fitting, significance calls, and LFC shrinkage.

Table 2: Agreement with R DESeq2 1.52.0 across six real-data designs. Each design was evaluated independently using a metric appropriate to the indicated pipeline stage.

Substep	Metric	Median	Worst
normalization	max rel. Δ	4.1e-15	5.7e-15
dispersion	p95 rel. Δ	8.5e-4	4.4e-3
glm_fit	p95 $\lvert\Delta\rvert$ (LFC)	2.4e-5	8.1e-5
significance	Jaccard@0.05	1.00000	0.99963
lfc_shrink	Pearson r	0.99999	0.99722

Every pipeline stage met its predefined tolerance across all six real-data designs. Normalization was effectively identical to R, and the largest dispersion discrepancy was 0.44% at the 95th percentile. Raw LFC estimates differed by at most 8.1×10^{-5} at the 95th percentile, and significant-gene sets had Jaccard similarity of at least 0.99963. Shrunken LFCs had Pearson correlation of at least 0.99722 with R.

3.2.1 Parity in the standard DE plots

In addition to summary statistics we validate against standard diagnostic plots used in DE analysis. Fig. 3 displays dispersion-estimates, MA and volcano plots from both pipelines. The gene-wise MLE cloud, the fitted trend, the shrunken MAP band and the 141 genes the outlier rule exempts from shrinkage all land in the same places, and the same 3,993 genes are called at padj < 0.05.

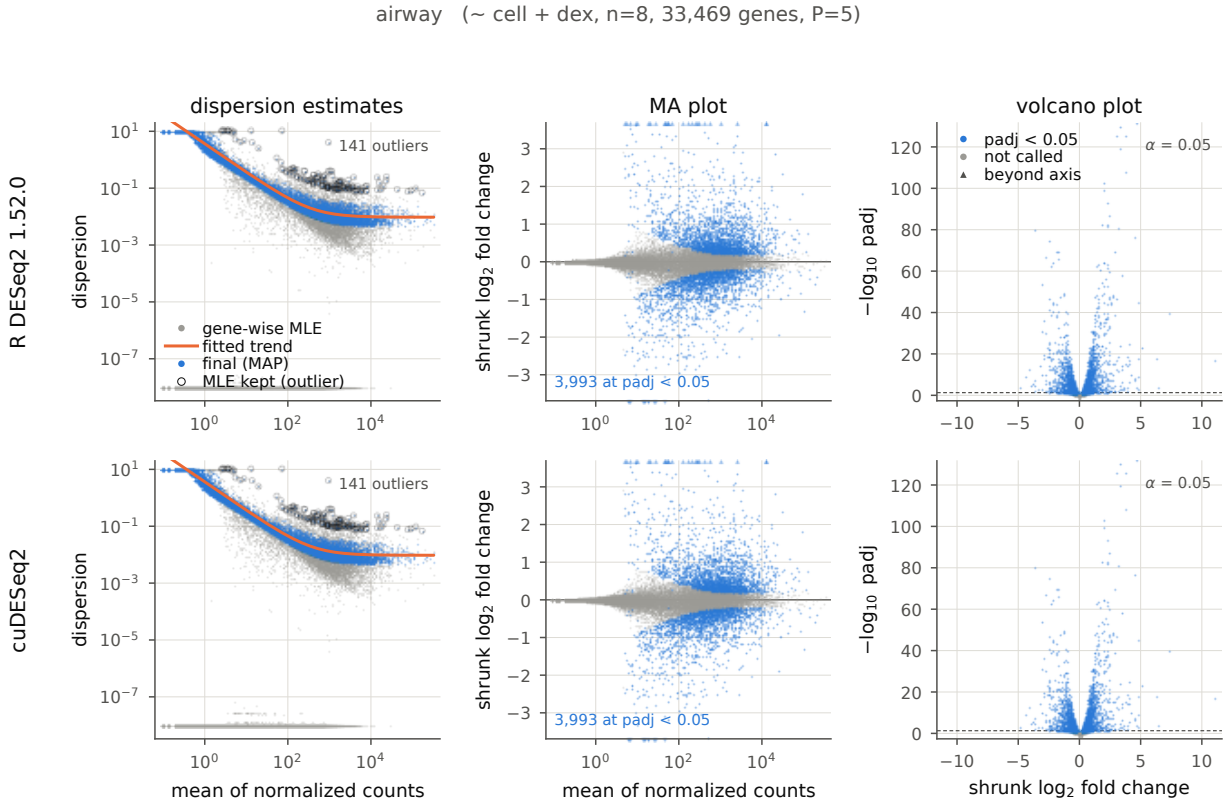

Figure 3: The three standard DESeq2 diagnostics on `airway` ($\sim$ cell + dex, n=8, P=5), drawn from R DESeq2 1.52.0 (top) and cuDESeq2 (bottom) by the same plotting code. *Left*, the `plotDispEsts` view: gene-wise MLE (grey), fitted trend (orange), final estimate (blue), and circled genes whose MLE exceeds the trend by more than two residual SDs and is therefore kept unshrunk. *Center*, MA plot of the apeGLM-shrunk LFC, with out-of-range genes on the boundary as in `plotMA`. *Right*, volcano plot. Axis limits are computed from R's output and reused for cuDESeq2. Reproduce: `validation/export_r_dispersion_details.R` then `validation/make_paper_figures.py`.

The two pipelines produce comprable top ranked genes Fig. 4. Each of the 14 logarithmically spaced N values shown, up to 2,000 and on every design, they share $\geq$99.6% of

the top N. Their called sets agree to a Jaccard index of $\geq$0.961 across the whole range $\alpha \in [10^{-3}, 0.2]$. At α=0.05, four of the six agree exactly and the other two reach 0.99963 and 0.99997.

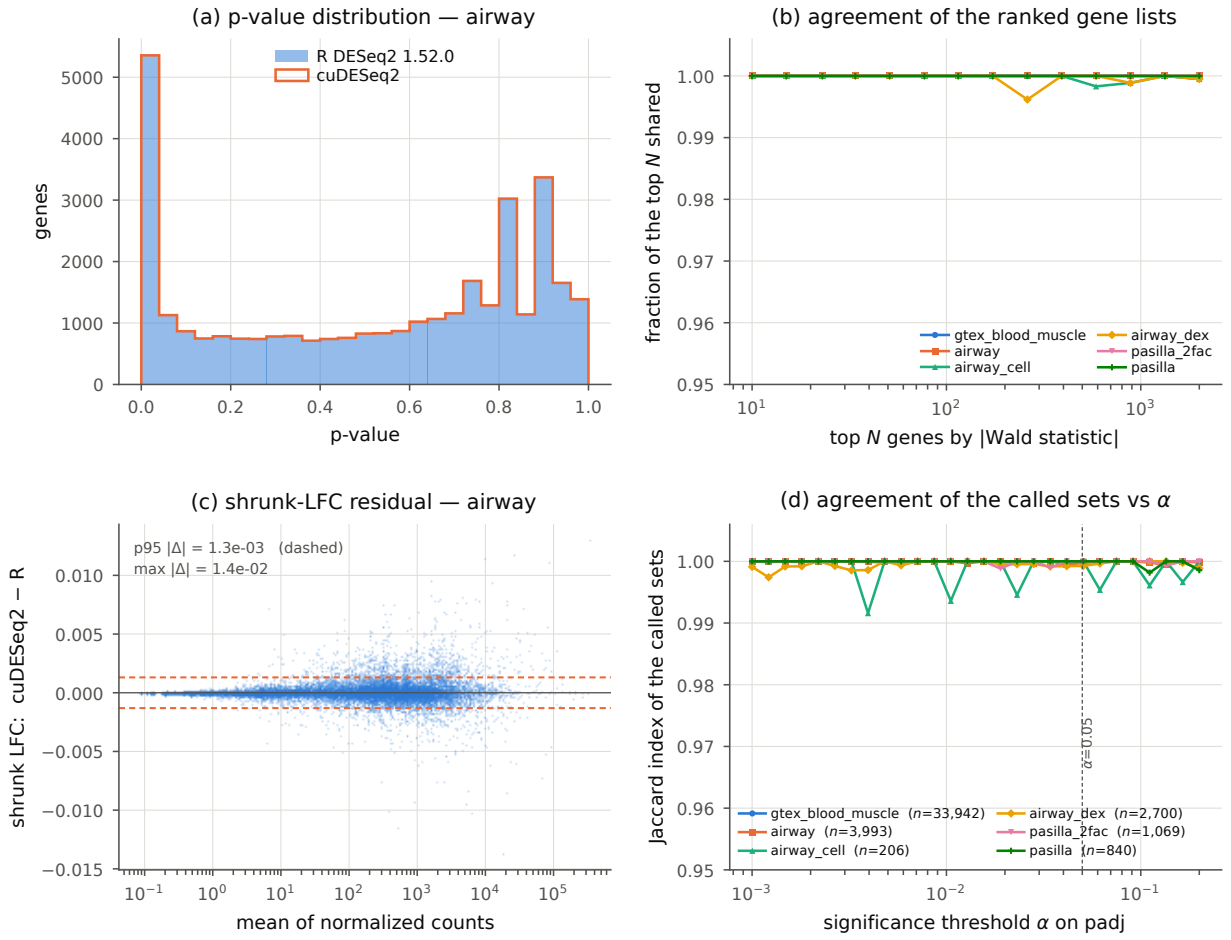

Figure 4: Quantitative agreement across all six designs. *(a)* p-value distributions on `airway`, overlaid. *(b)* Fraction of the top N genes shared by the two rankings, ranked by |Wald statistic| to avoid comparing tie-breaks among genes whose p-value has underflowed in R. *(c)* apeGLM-shrunk LFC residual against expression, showing no mean-dependent bias; dashed lines mark p95 $|\Delta|$. *(d)* Jaccard index of the called sets as the significance threshold moves, with the number of genes R calls at α=0.05 in the legend. The index is granular to $\sim 1/n$, so the small called sets move in visible steps. The `gtex` curve now includes count replacement and refitting in both pipelines.

3.3 Real-data timing versus R

We benchmark all 6 desgins to compare the end-to-end wall time of R DESeq2 1.52.0 with three GPU execution modes: eager, graph, and Triton. The graph and Triton modes only differe from eager cuDESEQ2 in the dispersion stage.

Table 3: End-to-end wall time on the real datasets: R 4.6.0/DESeq2 1.52.0 with one or 12 BiocParallel workers, and cuDESeq2 on one A100. The R runs and GPU runs use the standard pipeline; the `gtex` row includes count replacement and affected-gene refitting. Reproduce: `bench/run_r_parallel.R` → `bench/results/r_parallel_a100_12worker.json`; retained GPU timings: `bench/results/timings.json`.

			R DESeq2 (s)		cuDESeq2, one A100 (ms)		
Dataset	P	n	1 worker	12 workers	eager	graph	Triton
pasilla	2	7	9.4	7.3	1022	673	751
pasilla_2fac	3	7	10.2	7.4	1536	960	876
airway_dex	2	8	26.0	11.9	1330	1010	944
airway_cell	4	8	29.3	10.4	4260	3665	3629
airway	5	8	32.1	12.1	2578	1707	1531
gtex	2	300	276.0	63.4	8526	8284	4744

The best-mode speedup over the faster R configuration is 2.9–13.4×; `airway_cell` and `gtex` are the endpoints.

Triton is the fastest total mode on five of the six designs with graph being the fastest on `pasilla`. Triton has the fastest dispersion substep on all six: fusing the wider Cox–Reid solve (Sec. 2.2) cuts the dispersion substep by 8.7× on `pasilla_2fac` (P=3) and 2.7× on `airway` (P=5) against the eager loop (730→83 ms and 1586→583 ms, same run). `airway`'s dispersion substep also includes the CPU R-stream replay of Sec. 2.2 in every mode. We note that while it typically lags slightly to trition, the graph implementation is bit-identical to eager(Sec. 3.2).

4 Discussion

cuDESeq2 shows that the standard DESeq2 analysis pipeline can be moved to a GPU without replacing its statistical model or sacrificing quality. Across six real-data designs, the

Table 4: Per-substep wall time (ms) on the real datasets, one-worker R on the reference host vs. the three cuDESeq2 modes. The `glm_fit` timing includes the initial Wald fit and Cook's-distance replacement/refitting, but not the initial dispersion fit; **total** is the sum of the five stage medians. Reproduce: `bench/bench.py` → `bench/results/TABLES.md`.

Dataset	Substep	R	eager	graph	Triton
pasilla	dispersion	1760	417	71	20
	glm_fit	2035	39	39	39
	significance	223	62	61	62
	lfc_shrink	5314	504	501	630
	total	**9485**	**1022**	**673**	**751**
pasilla_2fac	dispersion	2134	730	163	83
	glm_fit	1465	334	338	335
	significance	204	75	60	59
	lfc_shrink	6043	396	399	398
	total	**9999**	**1536**	**960**	**876**
airway_dex	dispersion	4732	427	110	52
	glm_fit	5205	57	56	56
	significance	1391	145	126	124
	lfc_shrink	14810	699	716	711
	total	**26320**	**1330**	**1010**	**944**
airway_cell	dispersion	5668	753	185	60
	glm_fit	3850	38	36	37
	significance	863	115	115	119
	lfc_shrink	19755	3353	3327	3412
	total	**30326**	**4260**	**3665**	**3629**
airway	dispersion	7463	1586	719	583
	glm_fit	4438	62	61	61
	significance	895	118	119	120
	lfc_shrink	19488	811	808	767
	total	**32472**	**2578**	**1707**	**1531**
gtex	dispersion	184599	3507	3625	200
	glm_fit	51134	3148	2794	2694
	significance	492	613	610	609
	lfc_shrink	40117	1254	1252	1237
	total	**279604**	**8526**	**8284**	**4744**

`normalization` omitted (≤4 ms cuDESeq2 on every set); full versioned R records are in `bench/results/timings.json` and `bench/results/r_parallel_a100_12worker.json`. All GPU values come from the same A100 session.

implementation reproduced the intermediate quantities used by normalization, dispersion estimation, GLM fitting, significance testing, and LFC shrinkage (Table 2), while reducing end-to-end runtime by 2.9–13.4× relative to the fastest R DESeq2 run on the A100 benchmark host. These gains make repeated, reference-compatible differential-expression analyses more practical in settings such as model evaluation, where the same analysis is run many times.

The primary speedup comes from replacing sequential per-gene optimization with batched GPU computation. We obtain further speedup through CUDA-graph replay and Triton fusion which reduce the overhead of the dispersion iterations. Speedup varies across designs due to factors such as: the time spent in dispersion fitting, shrinkage, and the remaining CPU steps depends on sample size and the design matrix.

We note that our validated scope includes single-factor designs with up to four levels, additive multi-factor designs with up to P=5, and continuous covariates. The fused Triton path supports $P \in \{2, \ldots, 6\}$ and otherwise uses the eager implementation.

We note that in many instances numerical equivalence depends on subtle implementation details beyond published technical specifications. The dispersion prior, Wald standard errors, and apeGLM shrinkage each exposed behavior that is not fixed by the model specification alone (Sections 2.2, 3.2.1, and 2.4). Reproducing such details is sometimes necessary to obtain DESeq2 equivalent output. For example, we applied DESeq2's minmu floor when forming the GLM weight matrix used to calculate Wald standard errors; without it, genes with near-zero fitted counts receive different standard errors and can produce different p-values.

In other instances we achieve fidelity to the deseq2 method descrption with minor differences in floating-point rounding. For example Triton evaluates the same dispersion objective in a fused kernel, but its different floating-point reduction order and digamma implementation introduce small numerical differences from eager PyTorch. The maximum absolute eager–Triton dispersion difference retained by the benchmark is 0.2604 on GTEx

and at most 3.2×10^{-7} across the five smaller designs. In such instances, it is worth noting that these differences arise from implementation choices, not from a change to the DESeq2 model, and neither implementation is inherently closer to the intended calculation under that statistical model specification.

In conclusion, cuDESeq2 preserves the DESeq2 analysis while making repeated differential-expression analyses substantially faster on a GPU. Our hope is that this improvement will support the bioinformatics community by enabling reference-compatible DE analysis in workflows that require many repeated comparisons.

Funding

This work was funded by Synthesize Bio.

Conflict of Interest

All authors are affiliated with Synthesize Bio, which developed and maintains the software described here and released it under the MIT license. The authors declare no other competing interests.

Data and software availability

The pasilla benchmark uses the *Drosophila melanogaster* RNA-seq experiment distributed in the Bioconductor `pasilla` data package (`https://bioconductor.org/packages/pasilla`) (Brooks et al., 2011). The airway benchmark uses the human airway smooth-muscle-cell experiment in the Bioconductor `airway` data package (`https://bioconductor.org/packages/airway`) (Himes et al., 2014). The GTEx whole-blood versus skeletal-muscle benchmark uses recount2 gene-level data for SRA study SRP012682 (`https://recount-opendata.s3.amazonaws.com/recount2/v2/SRP012682/`

`rse_gene.Rdata`) (GTEx Consortium, 2020; Collado-Torres et al., 2017).

Source code, benchmark inputs, reference-generation scripts, and the Docker environment are available at `https://github.com/synthesizebio/gpu_deseq` under the MIT license.

References

A. K. Adduri, D. Gautam, B. Bevilacqua, et al. Predicting cellular responses to perturbation across diverse contexts with State. *bioRxiv*, 2025. doi: 10.1101/2025.06.26.661135.

Constantin Ahlmann-Eltze and Wolfgang Huber. glmGamPoi: fitting gamma–poisson generalized linear models on single cell count data. *Bioinformatics*, 36(24):5701–5702, 2020. doi: 10.1093/bioinformatics/btaa1009.

Angela N. Brooks, Li Yang, Michael O. Duff, Kasper D. Hansen, Jung W. Park, Sandrine Dudoit, Steven E. Brenner, and Brenton R. Graveley. Conservation of an RNA regulatory map between Drosophila and mammals. *Genome Research*, 21(2):193–202, 2011. doi: 10.1101/gr.108662.110.

Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with artificial intelligence: priorities and opportunities. *Cell*, 187(25):7045–7063, 2024. doi: 10.1016/j.cell.2024.11.015.

Leonardo Collado-Torres, Abhinav Nellore, Kai Kammers, et al. Reproducible RNA-seq analysis using recount2. *Nature Biotechnology*, 35(4):319–321, 2017. doi: 10.1038/nbt.3838.

Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.

GTEx Consortium. The GTEx consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020. doi: 10.1126/science.aaz1776.

Blanca E. Himes, Xiaofeng Jiang, Peter Wagner, Ruoxi Hu, Qiyu Wang, et al. RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLoS ONE*, 9(6):e99625, 2014. doi: 10.1371/journal.pone.0099625.

Wenxing Hu, Haotian Zhang, Yu H. Sun, Shaolong Cao, Jake Gagnon, Yuka Moroishi, Yirui Chen, Zhengyu Ouyang, and Baohong Zhang. ScaleSC: a superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU. *Bioinformatics Advances*, 5(1):vbaf167, 2025. doi: 10.1093/bioadv/vbaf167.

G. Koytiger, A. M. Walsh, V. Marar, et al. Generative genomics accurately predicts future experimental results. *bioRxiv*, 2025. doi: 10.1101/2025.09.08.674753.

Michael I. Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014. doi: 10.1186/s13059-014-0550-8.

Boris Muzellec, Maria Teleńczuk, Vincent Cabeli, and Mathieu Andreux. PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics*, 39(9): btad547, 2023. doi: 10.1093/bioinformatics/btad547.

Yusuf H. Roohani, Tony J. Hua, Po-Yuan Tung, et al. Virtual Cell Challenge: toward a Turing test for the virtual cell. *Cell*, 188(13):3370–3374, 2025. doi: 10.1016/j.cell.2025.06.008.

Christina V. Theodoris, Ling Xiao, Anant Chopra, et al. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, 2023. doi: 10.1038/s41586-023-06139-9.

Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23(2):451–464, 2026. doi: 10.1038/s41592-025-02980-0.

Anqi Zhu, Joseph G. Ibrahim, and Michael I. Love. Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. *Bioinformatics*, 35(12):2084–2092, 2019. doi: 10.1093/bioinformatics/bty895.

Supplementary benchmark details

The benchmark host was a KVM virtual machine with an Intel Xeon CPU at 2.20 GHz, 6 physical cores and 12 logical CPUs, 83 GiB RAM, and Linux 5.10. GPU measurements used one NVIDIA A100-SXM4-40GB. The software environment was PyTorch 2.6.0 with CUDA 12.4 and NVIDIA driver 550.90.07; the R environment was R 4.6.0 with DESeq2 1.52.0 and apeglm 1.34.0. OMP, OpenBLAS, and MKL thread counts were pinned to one for the R measurements. Every reported timing is the median of five measured repetitions following an untimed warm-up.

Supplementary algorithms

Supplementary Algorithm S1 gives a high-level view of the fused Triton dispersion fit sketched in Fig. 2; Supplementary Algorithm S2 gives the full kernel, including the Armijo line search, the step-size adaptation, and the Cox–Reid objective/gradient routine shared by the CR–MLE and MAP passes.

Supplementary Algorithm S1 Fused Triton dispersion fit (high-level)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G\times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G\times S}$; design $\mathbf{X} \in \mathbb{R}^{S\times P}$; initial dispersions $\boldsymbol{\alpha}^{(0)}$; optional MAP prior.
Ensure: optimized log dispersions $\mathbf{a}$.

Launch one Triton program for every gene g in parallel.
  Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
  Initialize $a \leftarrow \mathrm{clip}(\log \alpha_g^{(0)}, -30, 10)$ and step size $\kappa \leftarrow \kappa_0$.
  repeat inside the same fused program
    Compute the negative-binomial log-likelihood and Cox–Reid adjustment.
    If fitting MAP dispersion, add the Gaussian prior on $a = \log \alpha$.
    Compute the gradient $q \leftarrow \partial \ell / \partial a$.
    Propose the gradient-ascent step $\widetilde{a} \leftarrow a + \kappa q$.
    Accept or reject $\widetilde{a}$ using the Armijo condition; adapt κ.
  until converged or *maxit* is reached.
  Store the final a_g once in global memory.

Supplementary Algorithm S2 Detailed fused Triton dispersion kernel (CR–MLE or MAP)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G\times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G\times S}$; design $\mathbf{X} \in \mathbb{R}^{S\times P}$, $P \in \{2,\dots,6\}$; initial dispersions $\boldsymbol{\alpha}^{(0)} \in \mathbb{R}_+^{G}$; compile-time $h \equiv$ HasPrior; prior means $\mathbf{m}$ and variance σ^2 when $h = 1$; initial step bound κ_0.
Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

parallel for Triton program $g \in \{0,\dots,G-1\}$ **do** (one program per gene)
  Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
  Set $a \leftarrow \mathrm{clip}\Big(\log \alpha_g^{(0)}, -30, 10\Big)$, $\kappa \leftarrow \kappa_0$, $n_{\mathrm{acc}} \leftarrow 0$, and $c \leftarrow 0$.
  Set $(\ell, q) \leftarrow \mathrm{ObjectiveGradient}(a, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$.
  while $c <$ *maxit* and not converged **do**
    $c \leftarrow c + 1$ and propose $\widetilde{a} \leftarrow a + \kappa q$.
    If $q \neq 0$ and $\widetilde{a} \notin [-30, 10]$, shorten κ so that $\widetilde{a}$ lies on the violated boundary.
    Set $(\widetilde{\ell}, \_) \leftarrow \mathrm{ObjectiveGradient}(\widetilde{a}, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$.
    Set $r \leftarrow [\,\widetilde{\ell} \geq \ell + \varepsilon\kappa q^2\,]$. (Armijo acceptance flag)
    Select $a' \leftarrow r\,?\,\widetilde{a} : a$, $\ell' \leftarrow r\,?\,\widetilde{\ell} : \ell$, and $\Delta \leftarrow \ell' - \ell$.
    Set $(\_, q') \leftarrow \mathrm{ObjectiveGradient}(a', \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ and $q \leftarrow r\,?\,q' : q$.
    Set $n'_{\mathrm{acc}} \leftarrow n_{\mathrm{acc}} + r$ and $\kappa_{\mathrm{acc}} \leftarrow \min(1.1\kappa, \kappa_0)$.
    If r and $n'_{\mathrm{acc}} \bmod 5 = 0$, set $\kappa_{\mathrm{acc}} \leftarrow \kappa_{\mathrm{acc}}/2$.
    Set $\kappa \leftarrow r\,?\,\kappa_{\mathrm{acc}} : \kappa/2$, $n_{\mathrm{acc}} \leftarrow n'_{\mathrm{acc}}$, $a \leftarrow a'$, and $\ell \leftarrow \ell'$.
    Converge if $r \wedge (\Delta < 10^{-6} \ \vee \ a < \log(\mathit{minDisp}/10))$.
  end while
  Store $a_g \leftarrow a$ and $c_g \leftarrow c$ in global memory.
end parallel for
function ObjectiveGradient$(a, \mathbf{y}, \boldsymbol{\mu}, \mathbf{X}, h, m, \sigma^2)$
  Set $\alpha \leftarrow e^a$, $\mathbf{W} \leftarrow \mathrm{diag}(\boldsymbol{\mu}/(1 + \alpha\boldsymbol{\mu}))$, and $\mathbf{B} \leftarrow \mathbf{X}^{\mathsf{T}}\mathbf{W}\mathbf{X}$.
  Compute $\ell \leftarrow \ell_{\mathrm{NB}}(\mathbf{y}; \boldsymbol{\mu}, \alpha) - \frac{1}{2}\log\det(\mathbf{B})$.
  Compute $q \leftarrow \partial\ell/\partial a$, including $\mathrm{tr}\big(\mathbf{B}^{-1}\partial\mathbf{B}/\partial\alpha\big)$ via an unrolled 2×2 formula (P=2) or $P\times P$ LU solve.
  if $h = 1$ **then** $\ell \leftarrow \ell - (a - m)^2/(2\sigma^2)$ and $q \leftarrow q - (a - m)/\sigma^2$. (MAP only)
  return (ℓ, q).
end function